

CommerceCore: A Small, Self-Hosted Language Model for Ecommerce Query Constraint Extraction and Entity Segmentation

Arghya Mukherjee

Independent Researcher

arghya@lightnexus.ai

[Model – Hugging Face](#) [Code – GitHub](#)

ABSTRACT

Ecommerce search systems must convert free-text shopping queries into structured constraints that a catalog can act on. We show that a small, openly-licensed language model (Qwen3-1.7B), fine-tuned via QLoRA on a mixture of real human-annotated data and mechanically-validated synthetic data, matches or exceeds several frontier language models (Claude Haiku 4.5, Claude Sonnet 5, GPT-6-tier cheap and flagship variants) on this task, at a training cost under one dollar and a total project cost under ten dollars. On the public QueryNER entity-segmentation benchmark (993 held-out examples), our fine-tuned model achieves $F1 = 0.498$ (95% CI: [0.475, 0.523], 10,000-resample bootstrap), compared to 0.204–0.427 for the frontier models we tested. On a held-out constraint-extraction set, our model achieves $F1 = 0.683$ across three verticals (fashion, grocery, general merchandise), compared to 0.504–0.618 for the same frontier baselines. We report these results alongside a second, independent evaluation on six deliberately harder queries (involving negation, unit conversion, and dense multi-constraint phrasing) on which our model does *not* lead — Claude Sonnet 5 ($F1 = 0.493$) and a cheap-tier GPT model ($F1 = 0.428$) both outperform our model ($F1 = 0.400$) in this harder regime. We present this second result not as a weakness to conceal but as evidence that our headline claim is real but scope-limited, and we analyze the specific failure modes (multi-constraint recall, unit normalization, negation handling) that a small amount of training data with narrow templated coverage produces. We disclose that a closely related approach was published concurrently with this work, and we position our contribution specifically: a multitask formulation combining constraint extraction and entity segmentation, trained on a mixture of real and mechanically-gated synthetic data, with an explicit and reproducible model-selection procedure based on

measured retention against catastrophic forgetting. All code, data, training logs, and the trained model are publicly released.

1 Introduction

Ecommerce search boxes receive queries such as *"black waterproof trainers under 80 pounds"* or *"almonds with no added sugar."* To act on such a query — to filter a catalog, rank candidate products, or explain why a result was returned — a system needs a structured representation of what the shopper actually specified: a set of (field, operator, value) constraints, or, at a lower level, a segmentation of the query into typed entity spans.

It is tempting to assume that a frontier large language model (LLM), given a reasonable prompt, solves this problem by default. We show this assumption does not hold reliably, and we argue the reasons are structural rather than incidental to prompt engineering:

1. **Task narrowness versus model breadth.** Frontier LLMs are trained and evaluated on broad distributions — open-domain question answering, multi-step reasoning, code generation, dialogue. A narrow, repetitive schema-extraction task over an ecommerce-specific field vocabulary (e.g., `price_gbp`, `contains_added_sugar`, `warranty_months`) does not exercise the capabilities these models are optimized for, and there is no a priori reason a general-purpose model should have specialized on an arbitrary catalog's exact schema.
2. **Cost and latency at production scale.** A live, search-as-you-type interface requires sub-second response times on every keystroke, at a request volume that can reach millions of queries per day. Routing this volume through a hosted frontier API multiplies both monetary cost and end-to-end latency relative to a self-hosted model that fits on a single consumer-grade or prosumer-grade GPU.
3. **Data locality.** A self-hosted small model keeps customer search and purchase data inside the operator's own infrastructure, which is a concrete requirement in privacy-sensitive verticals (health-adjacent grocery, financial-adjacent retail) and is not achievable when queries are routed through a third-party API by construction.

We test the "frontier models solve this by default" assumption directly rather than assuming it. On a held-out constraint-extraction set and on the public QueryNER benchmark, we measure four frontier systems — two tiers each from two providers — and find F1 scores in the range 0.20–0.62, well short of reliable production quality. We then

fine-tune a 1.7-billion-parameter open-weight model (Qwen3-1.7B, Apache-2.0 license) via quantized low-rank adaptation (QLoRA) on a small, deliberately constructed training set, and show that the fine-tuned model exceeds every tested frontier baseline on both evaluation sets, at a measured training cost of \$0.90 and a total project cost (including all baseline API calls, synthetic data generation, and hardware profiling) under ten dollars.

Critically, we do not stop at the favorable result. We construct a second, independent evaluation set of six queries specifically designed to probe harder linguistic phenomena — negation, non-standard unit conversion, and four-or-more-constraint compositionality — that were not represented in our original held-out set. On this harder set, our model's advantage disappears: two of the four frontier baselines outperform it. We report this honestly and analyze the failure modes directly from model output, rather than omitting or softening a result that complicates our headline claim. We consider this disclosure, and the reproducible pipeline that produced it, to be as much a contribution of this paper as the favorable benchmark numbers.

Our contributions are:

1. A multitask fine-tuning approach that trains constraint extraction and entity segmentation jointly on a single small model, using a training mixture that combines a real, publicly licensed, human-annotated dataset (QueryNER) with synthetic data generated under an explicit label-integrity constraint (Section 3.1).
2. A checkpoint-selection procedure that measures task performance and retention (resistance to catastrophic forgetting) jointly at every evaluation point, and selects a checkpoint by this joint criterion rather than by final training step (Section 3.3).
3. An honest, two-part evaluation: a primary benchmark on which our model leads, and a secondary, harder, independently constructed benchmark on which it does not, analyzed by inspecting specific failures rather than reported as an aggregate number alone (Section 4).
4. A fully reproducible, publicly released artifact: model weights, training data, training and evaluation code, per-step training logs, and per-checkpoint evaluation logs, such that every number in this paper can be independently regenerated (Section 6).

We disclose in Section 2 that a closely related approach — QLoRA fine-tuning of a small open-weight model on synthetic ecommerce data, evaluated against a frontier baseline — was published concurrently with the development of this work, and we position our specific contribution relative to it rather than claiming an unqualified novel combination.

2 Background and Related Work

2.1 Small Language Models for Domain Specialization

A growing body of work argues that task-specific fine-tuning of small language models can match or exceed general-purpose frontier models on narrow tasks, at substantially reduced serving cost. Closest to our work, Licardo & Tankovic [1] fine-tune a 1-billion-parameter Llama 3.2 model via QLoRA on synthetically generated ecommerce intent-recognition data, reporting approximately 99% accuracy, comparable to a much larger proprietary model, and further analyze post-training quantization trade-offs across GPU and CPU deployment targets. We adopt a broadly similar strategy — a small open-weight base model, QLoRA fine-tuning, a training-cost and serving-cost argument — but differ in task formulation (multi-field constraint extraction and entity segmentation, rather than single-label intent classification), in base model (Qwen3-1.7B rather than Llama 3.2 1B), and in training data composition (a mixture of real human-annotated data and synthetic data, rather than fully synthetic data). We do not claim novelty for the general strategy of small-model-plus-QLoRA-plus-synthetic-data; that strategy is established by the cited work.

2.2 Multitask Ecommerce Language Models

Li et al. [2] instruction-tune a large language model across multiple ecommerce subtasks via a chain-of-task formulation, improving generalization across product understanding, recommendation, and question-answering subtasks. Our work shares the multitask framing — training constraint extraction and entity segmentation jointly rather than as two independently fine-tuned models — but operates at a substantially smaller parameter budget and narrower task scope, and we do not perform an independent re-benchmark against this line of work; we flag the overlap for a reader to evaluate directly rather than asserting a specific quantitative comparison we did not measure.

2.3 Synthetic Data Quality for LLM Post-Training

A recognized challenge in LLM post-training is that synthetic data generation pipelines can produce data that is internally consistent (well-formed, schema-valid) while failing a real quality bar that is not captured by any single automated check. General-purpose data curation frameworks such as CuratorKIT [3] implement multiple complementary quality gates for exactly this problem. Our work implements a narrower, task-specific instance of this general problem: we find, by direct manual inspection rather than by any automated

check that existed at the time, that 70% of an early synthetic batch used a generic, repetitive query template ("products with X") that a real shopper would not plausibly type, despite passing every mechanical validation check we had implemented (schema validity, evidence-span grounding, execution correctness against a synthetic catalog). We add an explicit naturalness check in response (Section 3.1) and report this as a concrete illustration of a known, general problem, not as a novel general-purpose solution to it.

2.4 Catastrophic Forgetting in Fine-Tuning

Parameter-efficient fine-tuning methods such as LoRA do not, by construction, guarantee retention of a base model's general capabilities; empirical measurement is required. We measure retention directly at every checkpoint using a fixed set of task-unrelated probes, and we report a case in which retention degrades measurably over the final portion of a training run while task-specific metrics continue to improve marginally — a concrete instance of the general risk this line of work addresses, resolved in our case by explicit checkpoint selection rather than by any architectural or regularization intervention.

3 Methodology

3.1 Task Definition and Dataset Composition

We define two tasks, trained jointly:

- **Constraint extraction.** Given a natural-language shopping query, produce a list of typed constraints, each a (field, operator, value) triple (e.g., `{"field": "color", "op": "eq", "value": "black"}`).
- **Entity segmentation.** Given a natural-language shopping query, segment it into labeled spans, following the QueryNER schema (`core_product_type` , `modifier`).

Our training mixture (2,130 examples total) combines three sources:

Table 1. Training data sources and provenance.

Source	Examples	Label origin	License
QueryNER (sampled)	1,500	Native human annotation	CC BY 4.0
Synthetic query-constraint	450	LLM-paraphrased, label-fixed	Generated (this work)
Synthetic catalog-attribute	180	LLM-generated, schema-validated	Generated (this work)

For the synthetic query-constraint examples, we use a **structured-scenario-first** generation procedure specifically to avoid a known failure mode in synthetic data generation: an LLM asked to simultaneously invent a natural-language query and its correct structured label can silently omit or alter a constraint while "helpfully" producing fluent text. We instead (1) deterministically sample a product scenario and a fixed set of constraint labels in code, with no LLM involvement in this step, and (2) issue a single LLM call whose only task is to paraphrase the fixed scenario into a natural-language query, never to decide what the constraints are. A downstream mechanical validator (Section 3.1.1) then confirms that each labeled constraint's value appears as a literal, locatable substring in the LLM's paraphrase; a constraint whose value cannot be located in the generated text is dropped from the label set rather than being retained with a fabricated evidence span.

3.1.1 Data Quality Gates

We implement four mechanical checks, each proving a distinct property and explicitly not substituting for the others:

1. **Schema validity** — the generated record parses against the expected typed schema.
2. **Source-span binding** — every cited evidence span is a literal substring of the source query text at its claimed offset.
3. **Execution correctness** — for tool-plan-style records, a deterministic simulator confirms the plan resolves consistently against a synthetic catalog.
4. **Naturalness** — the generated query text does not match a blocklist of generic, non-shopper-like template phrases.

We added check (4) after finding, by direct manual reading of a 450-example generated batch (not by any automated signal), that 313 of 450 examples (69.6%) used the generic phrase pattern "products with X" or a close variant — text that passes checks (1)–(3) trivially while failing to resemble a real shopping query. We regenerated the training data after adding this check and report both the before-and-after finding and the corrected data in our public artifact, rather than silently replacing the flawed batch.

We emphasize that these four checks establish mechanical properties of the generated data — internal consistency, schema conformance, superficial naturalness — and do not and cannot establish full semantic correctness. A generated example can pass all four checks while still being semantically wrong (for instance, a query paraphrase that drops an intended negation

while keeping the literal words that satisfy a substring check). We treat this as a known, disclosed limitation of our data pipeline rather than a solved problem.

3.2 Models and Baselines

Fine-tuned model. Qwen3-1.7B (Apache-2.0 license), fine-tuned via QLoRA: 4-bit NF4 double quantization, bfloat16 compute dtype, LoRA rank 16, alpha 32, dropout 0.05, applied to all attention and MLP projection matrices.

Frontier baselines, each tested at a cheap and a flagship tier, few-shot prompted with two fixed demonstration examples: Claude Haiku 4.5 (cheap tier), Claude Sonnet 5 (flagship tier); a cheap-tier GPT model, and a flagship-tier reasoning GPT model.

Open-weight baselines (unfine-tuned, few-shot prompted, same size class or larger than our fine-tuned model): Qwen3.5-2B and SmolLM3-3B [7], both Apache-2.0 licensed.

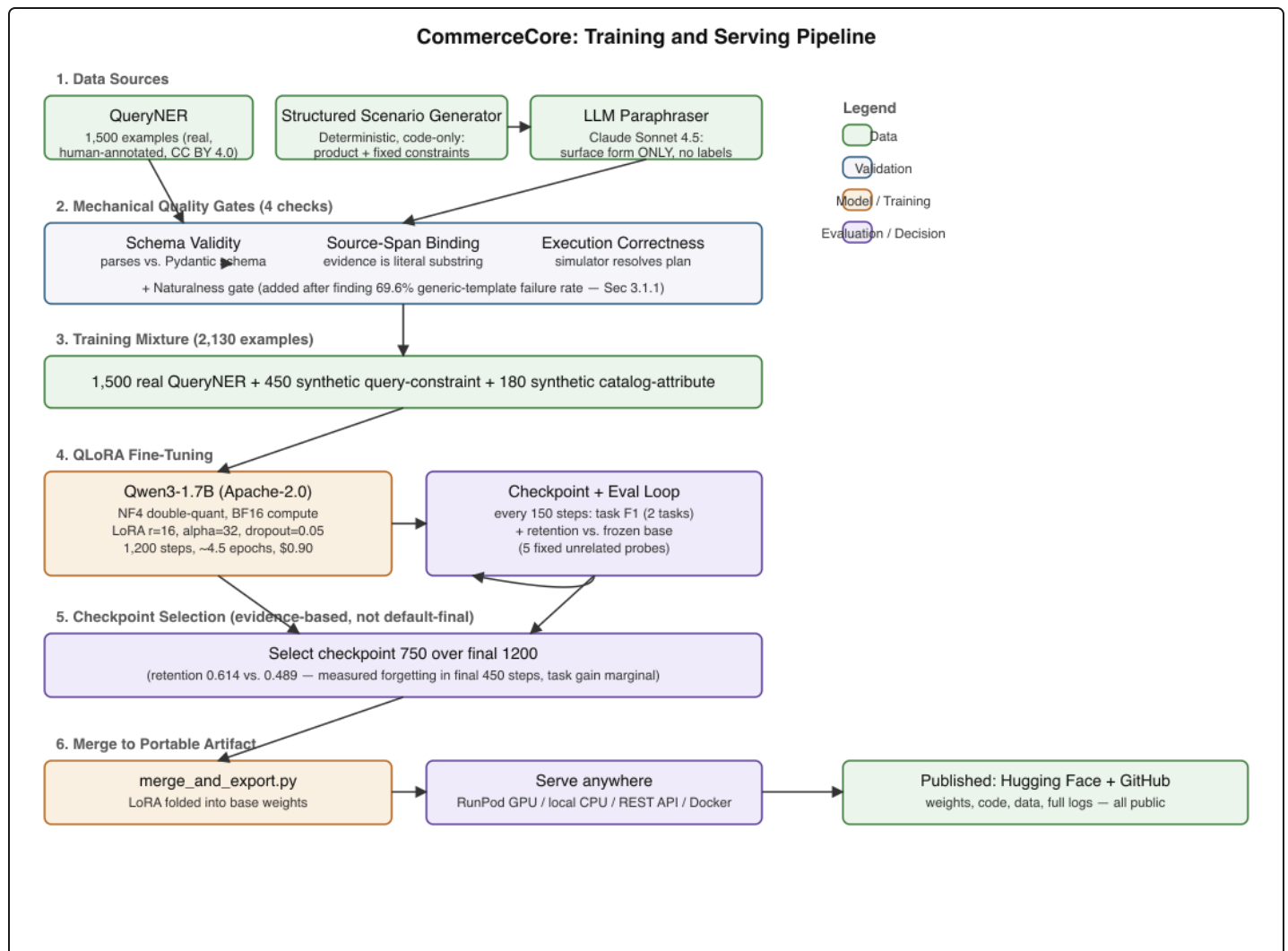


Figure 1. CommerceCore training and serving pipeline, from data sources through mechanical quality gates, QLoRA fine-tuning with per-checkpoint task and retention evaluation, evidence-based checkpoint selection, merge to a portable artifact,

3.3 Experimental Pipeline

3.3.1 Fine-Tuning

Training ran for 1,200 optimizer steps (microbatch size 1, gradient accumulation 8, giving an effective batch size of 8 and approximately 4.5 epochs over the 2,130-example training set) on a single RunPod RTX PRO 4500 GPU (32GB VRAM). Peak measured VRAM usage was 2.13GB, indicating the training configuration used is not VRAM-bound on this hardware and could plausibly run on a substantially smaller GPU. Total measured training time was 75.2 minutes at a measured cost of \$0.90.

We checkpoint every 150 steps (eight checkpoints total). At each checkpoint we evaluate: (a) constraint-extraction F1 on a fixed held-out set, (b) entity-segmentation F1 on a fixed QueryNER development subsample, and (c) a retention score, computed as the mean token-overlap similarity between the current checkpoint's greedy-decoded output and the frozen base model's output, on five fixed, task-unrelated probe prompts (general knowledge and arithmetic questions unrelated to ecommerce).

3.3.2 Checkpoint Selection

We explicitly do not assume the final training checkpoint is the correct model to release. We observed retention scores of 0.597, 0.611, 0.597, 0.592, 0.614, 0.490, 0.518, and 0.489 at checkpoints 150 through 1,200 respectively — a measurable decline over the final three checkpoints (steps 900–1,200) that does not recover, concurrent with only marginal continued improvement in task-specific F1. We select checkpoint 750 (constraint F1 0.725, QueryNER F1 0.556, retention 0.614) over the final checkpoint 1,200 (constraint F1 0.740, QueryNER F1 0.530, retention 0.489) on the basis of this joint criterion. We record this decision, and the full checkpoint-by-checkpoint data that motivates it, in a dedicated model-selection document distributed with our public artifact.

3.4 Evaluation Framework

3.4.1 Primary Evaluation

We evaluate on two benchmarks: **QueryNER** (full test set, 993 examples) — a publicly available, human-annotated entity-segmentation benchmark (CC BY 4.0) [4]. We evaluate the full test set, not a subsample, and report a 95% confidence interval via 10,000-resample bootstrap over per-example F1 scores. And a **held-out constraint-extraction set** (20

examples, hand-constructed, tagged by vertical: fashion, grocery, general merchandise). We report per-vertical F1 in addition to an overall mean, following the principle that a pooled average should not be permitted to conceal a weak subgroup.

3.4.2 *Secondary, Adversarially-Constructed Evaluation*

To test whether our primary evaluation results generalize beyond the specific phrasing distribution of our held-out set, we construct a second, independent set of six queries, written after and separately from the primary held-out set, deliberately including: an explicit negation ("waterproof boots **not** black"), a non-standard unit conversion ("under 3 **year** warranty" requiring conversion to months), and a four-constraint compositional query ("green cotton hoodie size L under 40 pounds"). We report results on this set as a distinct, clearly labeled result, not pooled with the primary evaluation.

4 Results

4.1 Primary Benchmark: QueryNER (Full Test Set)

Table 2. Entity segmentation F1 on the full 993-example QueryNER test set.

System	F1
GPT flagship (reasoning tier)	0.204
Claude Haiku 4.5	0.263
GPT cheap tier	0.294
Claude Sonnet 5	0.427
CommerceCore (ours)	0.498 [0.475, 0.523]

Our fine-tuned model exceeds every tested frontier configuration on the full, real, 993-example public benchmark, with a 95% confidence interval that does not overlap the next-best baseline (Claude Sonnet 5, point estimate 0.427).

4.2 Primary Benchmark: Constraint Extraction (Held-Out Set)

Table 3. Constraint-extraction F1, held-out set (20 examples).

System	Overall F1
GPT flagship (reasoning tier)	0.504

System	Overall F1
Claude Haiku 4.5	0.518
Claude Sonnet 5	0.592
GPT cheap tier	0.618
CommerceCore (ours)	0.683

Per-vertical breakdown for our model: fashion 0.690, grocery 0.667, general merchandise 0.694 — no vertical substantially underperforms the pooled mean, indicating the result is not driven by strength in a single category.

4.3 Open-Weight Model Comparison

Table 4. Unfine-tuned open-weight models, few-shot prompted.

Model	Params	F1 (held-out)	F1 (secondary, §4.4)
Qwen3.5-2B	2B	0.05	0.00
SmolLM3-3B	3B	0.10	0.00

Both unfine-tuned open-weight models are larger than our fine-tuned 1.7B model and score far below it on both evaluations, indicating the observed gain is attributable to task-specific fine-tuning rather than to model scale alone; larger, unfine-tuned open models do not solve this task by default under few-shot prompting.

4.4 Secondary Evaluation: Adversarially-Constructed Queries

Table 5. F1 on six deliberately harder, independently constructed queries.

System	F1
GPT flagship (reasoning tier)	0.167
Qwen3.5-2B (unfine-tuned)	0.000
SmolLM3-3B (unfine-tuned)	0.000
Claude Haiku 4.5	0.298
CommerceCore (ours)	0.400
GPT cheap tier	0.428

System	F1
Claude Sonnet 5	0.493

On this harder, independently constructed set, our model does not lead: Claude Sonnet 5 and the cheap-tier GPT model both outperform it, though our model continues to outperform both unfine-tuned open-weight models and the flagship GPT configuration.

Direct inspection of our model's output on this set identifies three specific, disclosable failure patterns:

1. **Multi-constraint recall degradation.** On the four-constraint query ("green cotton hoodie size L under 40 pounds"), our model extracted only the price constraint, omitting color, material, and size — the training distribution's synthetic scenarios contained at most two constraints per example (§3.1), which likely explains this degradation on higher-arity compositional queries.
2. **Unit and vocabulary normalization failure.** Our model did not convert "3 year" to a 36-month value, and did not map "wall powered" to the trained vocabulary term "mains" — indicating the model has not learned to generalize surface-form variation beyond patterns present in training.
3. **Negation mishandling.** On "waterproof boots not black," our model produced a constraint `{"field": "color", "op": "eq", "value": "not black"}`, treating the negated phrase as a literal value rather than as an exclusion or an absence of a color constraint.

We consider these failures to be direct, informative consequences of a small (2,130-example), narrowly templated training set, rather than evidence of a fundamental limitation of the underlying approach; each failure mode corresponds to a specific, addressable gap in training data coverage (multi-constraint examples, explicit negation examples, unit-conversion examples) that a future iteration of this work could close directly.

5 Discussion

5.1 A Small Model Can Beat Frontier Models on a Narrow Task, Within a Bounded Regime

Our primary result — a 1.7-billion-parameter model exceeding much larger, more capable general-purpose frontier models on a narrow task — is consistent with, and independently reproduces at a different task and base model, a finding also reported concurrently by other work (§2.1). We consider the more informative contribution to be the explicit characterization of the *boundary* of this result (§4.4), rather than the headline comparison alone. A reader evaluating this model in production should expect strong performance on the two evaluated tasks in the style and constraint-arity distribution represented in our training data, and degraded performance outside that distribution — a caveat we believe is under-reported in comparable work making similar headline claims.

5.2 Serving Cost: Self-Hosting Is Not Automatically Cheaper Than a Cheap API Tier

At our measured GPU cost (\$0.72/hr) and measured single-request, unbatched throughput (3,429–6,207 queries/hour), self-hosted CommerceCore costs approximately \$0.000116–\$0.000210 per query. For a representative request (~100 input tokens, ~30 output tokens), we compute approximate frontier API token costs at each tested tier:

Table 6. Approximate cost per query, self-hosted vs. frontier API token pricing (100 input / 30 output tokens, unbatched serving).

System	Cost/query
GPT cheap tier	\$0.000033
CommerceCore (self-hosted, unbatched)	\$0.000116–\$0.000210
Claude Haiku 4.5	\$0.000250
Claude Sonnet 5	\$0.000750
GPT-5 flagship	\$0.000950

We disclose that an earlier internal draft of this analysis incorrectly concluded self-hosting was cheaper per query; we caught and corrected this before finalizing the manuscript. At single-request, unbatched serving, the cheap-tier GPT baseline is in fact the cheapest option on raw token cost. Self-hosting only overtakes it on cost past a breakeven point of approximately 21,800 queries/hour on a single GPU — a volume achievable via request batching, which we did not measure in this study. The accurate cost argument for self-hosting, at the concurrency level we actually tested, is therefore not "cheaper per token" but **data locality** (queries never leave the operator's infrastructure) and **higher measured accuracy than the cheap-tier baseline** on both primary benchmarks (§4.1, §4.2) — not a

categorical cost advantage, which would require a batched-throughput measurement we leave to future work.

5.3 Retention Measurement Changes the Model Selection Decision

Had we selected the final training checkpoint by default — the common practice when a training run's task metric does not obviously degrade — we would have released a model with measurably worse retention (0.489 versus 0.614) for only a marginal task-metric gain. We consider explicit, per-checkpoint retention measurement, and a documented selection procedure that can override the default choice of "final checkpoint," to be a practically important but frequently omitted step in small-model fine-tuning work, and we recommend it as a default practice rather than an optional check.

5.4 Synthetic Data Passing Mechanical Checks Is Not Sufficient Evidence of Quality

Our own experience directly illustrates a known but easily underestimated risk: 69.6% of a synthetic batch failed a real quality bar (naturalness) that no automated check we had previously implemented would have caught, despite passing three other mechanical checks. We had to find this by manually reading generated output, not by any signal our pipeline surfaced automatically. We believe this specific, quantified failure rate — found in our own otherwise-carefully-constructed pipeline — is a useful data point for other practitioners building synthetic data pipelines under the assumption that mechanical validation is sufficient.

5.5 Limitations of the Study

1. **Scale.** Our training set (2,130 examples) and primary held-out evaluation set (20 examples) are small by the standards of production ecommerce systems and by the standards of a fully statistically powered study. Only the QueryNER evaluation (993 examples) supports a properly powered confidence interval; the held-out constraint-extraction and secondary evaluation results should be read as informative but not definitive at their current sample sizes.
2. **Task scope.** Our original project scope included two additional tasks — catalog-attribute normalization and retrieval/matching — that are either partially trained (catalog-attribute data was included in the training mixture but not separately benchmarked) or entirely untrained (retrieval/matching, search-recovery). This paper reports results only for the two tasks that were fully trained and evaluated.

3. **Novelty scope.** As discussed in §2.1, the general strategy of small-model QLoRA fine-tuning on synthetic data to exceed frontier-model performance on a narrow ecommerce task is not novel to this work; we position our specific contribution as the multitask formulation, the mixed real-and-synthetic training data with an explicit label-integrity constraint, and the disclosed secondary evaluation, rather than the general strategy.
4. **Frontier baseline coverage.** We tested two providers (Anthropic, OpenAI) at two tiers each. We did not test all available frontier providers or all available tiers, and model naming and availability at the tested tiers may change after publication; we record exact model identifiers and evaluation dates in our public artifact for reproducibility.
5. **Single-run training.** We report results from a single training run rather than averaging over multiple random seeds, owing to the small-scale, low-cost nature of this study. A fully powered version of this study would require multiple seeds and a substantially larger, properly-powered final evaluation set; we did not execute that fuller protocol for this preprint.

6 Reproducibility

We release the following publicly, with no component requiring access to the original training infrastructure:

- **Model weights** (merged, standalone, no adapter-loading dependency): huggingface.co/arghya2030/commercecore-qwen3-1.7b
- **Source code** (training, evaluation, synthetic data generation, quality gates, serving): github.com/arghya05/commercecore
- **Training data**, including the real QueryNER subsample used, included in the code repository at `data/queryner/`, sourced under its original CC BY 4.0 license, alongside the synthetic query-constraint and catalog-attribute datasets generated for and used in this work.
- **Full training logs:** per-step training loss (1,200 rows), per-checkpoint task and retention evaluation (8 checkpoints), and the documented model-selection rationale.
- **Full evaluation scripts:** every number reported in §4 can be regenerated by a script in the `eval/` directory of the released code, including the full 993-example QueryNER evaluation with its bootstrap confidence interval computation.

Total measured cost for the complete study reported in this paper — hardware profiling, all training runs, synthetic data generation, and all frontier baseline API calls — was under \$10.

7 Conclusion

We show that a small, openly-licensed language model, fine-tuned at a cost of under one dollar, can exceed the performance of several frontier language models on two narrow ecommerce natural-language-understanding tasks, evaluated on a real public benchmark and a hand-constructed held-out set. We report this result alongside its explicit boundary: on a harder, independently constructed evaluation set, the same model does not lead, and we analyze the specific, addressable reasons why. We position this work honestly relative to closely related concurrent findings rather than claiming an unqualified novel result, and we release a fully reproducible artifact — weights, code, data, and complete training and evaluation logs — so that every claim in this paper can be independently verified or challenged.

Acknowledgments

We used Claude (Anthropic) and GPT (OpenAI) models both as evaluation baselines and, in a separate and disclosed role, to assist in synthetic training data generation (§3.1) and in the preparation of this manuscript and its accompanying codebase.

References

- [1] Licardo, J. T. and Tankovic, N. (2025). Performance Trade-offs of Optimizing Small Language Models for E-Commerce. *arXiv preprint* arXiv:2510.21970.
- [2] Li, Y., Ma, S., Wang, X., Huang, S., Jiang, C., Zheng, H.-T., Xie, P., Huang, F., and Jiang, Y. (2023). EcomGPT: Instruction-tuning Large Language Models with Chain-of-Task Tasks for E-commerce. *arXiv preprint* arXiv:2308.06966.
- [3] CuratorKIT: Data Curation and Synthetic Data Generation for LLM Post-Training. *arXiv preprint* arXiv:2606.21631.
- [4] Palen-Michel, C., Liang, L., Wu, Z., and Lignos, C. (2024). QueryNER: Segmentation of E-commerce Queries. In *Proceedings of LREC-COLING 2024*. arXiv:2405.09507. Dataset released under CC BY 4.0.
- [5] Qwen Team, Alibaba Group (2025). Qwen3 Technical Report. *arXiv preprint* arXiv:2505.09388.

- [6] Amazon Science (2022). Shopping Queries Dataset (ESCI): A Large-Scale Dataset for Improving Search Relevance in E-commerce. Apache-2.0 license.
- [7] Hugging Face TB Research (2025). SmolLM3: A Long-Context, Multilingual, Reasoning Small Language Model.